



POLITECHNIKA OPOLSKA

PROGRAMOWANIE SYSTEMOWE

Sygnały

Ćwiczenie Nr 3

Proszę napisać program wykorzystujący elementarne operacje Unix wykorzystującego sygnały.

Współbieżne znajdowanie liczb pierwszych

Proszę napisać program, który ma znajdować liczby pierwsze w zadanym przedziale [Zd,...,Zg]. Liczba pierwsza to liczba naturalna większa od 1, która ma dokładnie dwa dzielniki: 1 i samą siebie. Posty algorytm sprawdzania, czy dana liczba n jest liczbą pierwszą dany jest poniżej:

```
int pierwsza (int n)
// Funkcja zwraca 1 gdy n jest liczbą pierwszą
{ int i,j=0;
  for(i=2;i*i<=n;i++)
  {
    if(n%i == 0)
      return(0) ;
  }
  return(1);
}
```

Zadanie 3.1:

Obliczenia należy przyspieszyć dzieląc zakres [Zd,...,Zg] na **n** podprzedziałów [Zd(1),...,Zg(1)], [Zd(2),...,Zg(2)] ,..., [Zd(n),...,Zg(n)], gdzie n jest liczbą procesów obliczeniowych.

1. Program powinien uruchamiać się z trzema argumentami - początek przedziału (Zd), koniec przedziału (Zg) i liczba procesów obliczeniowych (n).
2. Proces macierzysty powinien uruchomić odpowiednią liczbę procesów obliczeniowych, przekazać im przedział poszukiwań (jako argumenty przy uruchomieniu procesu funkcją exec lub przez przesłanie sygnałów).
3. Każdy z procesów obliczeniowych niezależnie szuka liczby pierwszej w swoim przedziale [Zd(i),...,Zg(i)].
4. Każdą znalezioną liczbę pierwszą proces obliczeniowy wysyła do procesu macierzystego za pomocą sygnału. W polu si_int struktury siginfo_t przesyła wartość odnalezionej liczby pierwszej.
5. Proces obliczeniowy kończy się po przeszukaniu swojego przedziału i zwraca liczbę znalezionych liczb pierwszych procesowi macierzystemu.
6. Proces macierzysty biera znalezione liczby pierwsze od procesów obliczeniowych. Drukuje je na ekranie i czeka na zakończenie procesów obliczeniowych. Wyświetla liczbę znalezionych liczb pierwszych w całym zakresie. **Proszę sprawdzić czy proces macierzysty odebrał wszystkie liczby pierwsze** (porównując liczbę odebranych sygnałów oraz sumę wartości otrzymanych przy zakończeniu procesów potomnych).

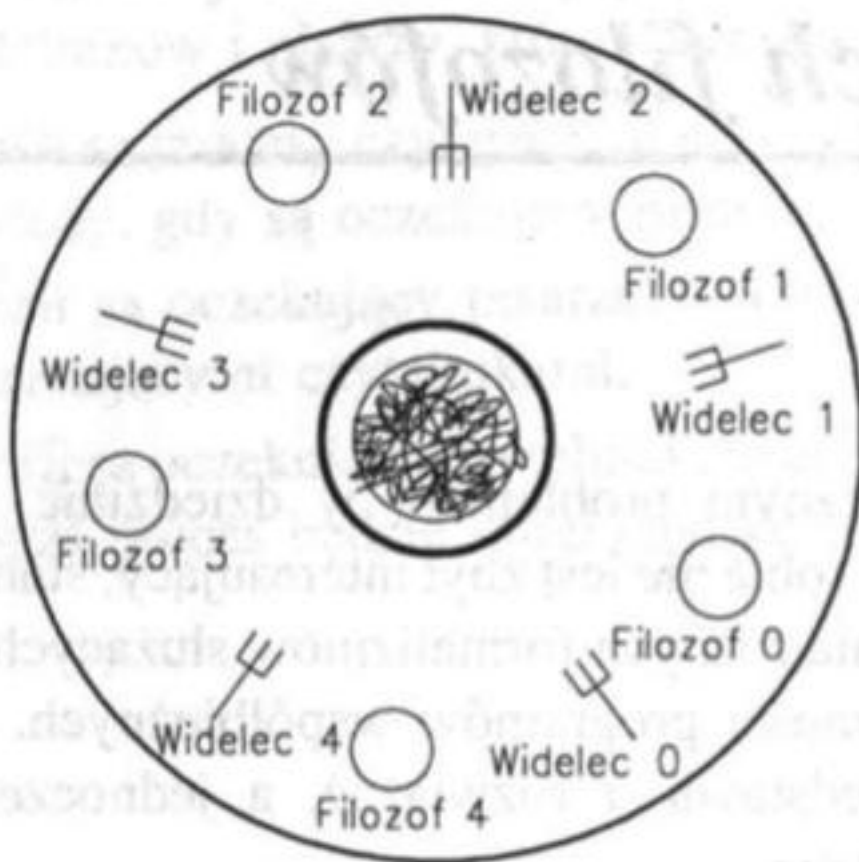
Zadanie 3.2:

Problem 5 uczujących filozofów

Proszę zaproponować rozwiązanie problemu 5 uczujących filozofów z zastosowaniem „kelnera”, przy założeniu, że zachowanie każdego z filozofów będzie identyczne.

Założenia problemu:

- Filozof może być w stanach „myśli”, „głodny” oraz „je”
- Filozof je tylko wtedy, gdy ma dwa widelce.
- Dwu filozofów nie może jednocześnie trzymać tego samego widelca.
- Nikt nie może być zagłodzony.
- Kelner decyduje, którzy z głodnych filozofów otrzymają widelce.



1. Proces macierzysty powinien uruchomić pięć procesów filozof.
2. Proces filozof powinien działać w następującej pętli (a-e):
 - a) Każdy proces filozof przez losowy czas (max 5s) jest w stanie „myśli” i drukuje to na ekranie:
filozof_X: myśli
 - b) Po zakończeniu stanu myśli proces filozofa zmienia stan na „głodny” i prosi kelnera o pozwolenie zajęcia widelców (wysyła sygnał do kelnera) i drukuje na ekranie:
filozof_X: głodny
i czeka na sygnał od kelnera.
 - c) Po otrzymaniu zgody na zajęcie widelców (sygnał otrzymany od kelnera) proces filozofa zajmuje widelce i drukuje na ekranie:
filozof_X: zajmuje widelce: A i B
 - d) Proces filozofa zmienia stan na „je” na losowy czas (max 5s) i drukuje na ekranie:
filozof_X: je
 - e) Po zakończeniu stanu „je” proces filozofa zwalnia widelce oraz informuje to tym kelnera (wysyła sygnał do kelnera) i drukuje na ekranie:
filozof_X: zwalnia widelce: A i B
gdzie: X, A, B to numer filozofa i numery widelców np.:
filozof 2 zajmuje widelce 2 i 3
3. Proces kelner powinien działać następująco:
 - Proces kelner oczekuje na sygnały od procesów filozofów
 - Po otrzymaniu sygnału zwolnienia widelców przez filozofa, proces kelner uaktualnia stan widelców (wolny, zajęty) i stan filozofa na myśli.
 - Po otrzymaniu sygnału prośby na zajęcie widelców od filozofa proces kelnera zmienia stan filozofa na „głodny”.
 - Po każdym otrzymaniu sygnału proces kelnera sprawdza czy któryś z procesów filozofa jest w stanie „głodny”. Jeśli istnieje proces filozofa w stanie „głodny” to sprawdza dla niego dostępność widelców. Jeśli widelce są dostępne to wysyła sygnał do filozofa o zgodzie na zajęcie widelców, uaktualnia stan widelców (wolny, zajęty) oraz zmienia stan filozofa na „je”.

przydatne funkcje:

https://man7.org/linux/man-pages/dir_all_alphabetic.html

<https://linux.die.net/>

```
#include <signal.h>
int sigfillset(sigset_t *set);
int sigdelset(sigset_t *set, int signo);
int sigemptyset(sigset_t *set);
int sigaddset(sigset_t *set, int signo);
int sigprocmask(int how, const sigset_t *_Nullable restrict set,
                sigset_t *_Nullable restrict oldset);

sighandler_t signal(int signum, sighandler_t handler);
int sigaction(int signum,
               const struct sigaction *_Nullable restrict act,
               struct sigaction *_Nullable restrict oldact);

int kill(pid_t pid, int sig);
int sigqueue(pid_t pid, int sig, const union sigval value);

int sigsuspend(const sigset_t *mask);
int sigwait(const sigset_t *restrict set, int *restrict sig);

#include <unistd.h>
unsigned int alarm(unsigned int seconds);
int pause(void);
```